

Spring Core

Interview Q&A with Layman Explanations • 2.5+ YOE

Each question has two parts: a **plain-words explanation** (understand the concept) and a **ready-to-say interview answer** (green box). Read the plain words to learn it, memorize the green box to deliver it.

Section 1 • Dependency Injection & Beans

Q1. Why choose constructor injection over field injection?

In plain words: Constructor injection means you get all your tools handed to you the moment you're created — like a chef who can't start until the kitchen is fully stocked. This guarantees nothing is missing. It also lets you lock those tools so nobody swaps them later (final fields), and makes testing easy because you can hand in fake tools directly. Field injection just slaps tools onto you after the fact — you can't lock them, and testing is harder because you need Spring's help to insert them.

Say this in the interview: Constructor injection lets me mark fields **final** (immutable), guarantees the dependency exists at object creation so there are no half-built objects, and makes unit testing easy since I can pass mocks straight through the constructor. Field injection can't do final, hides dependencies, and needs reflection to test.

Q2. A singleton bean has a field 'int counter'. Two users increment it at once. Problem? Fix?

In plain words: A singleton is ONE shared object used by everybody at the same time. Imagine one shared whiteboard and two people writing on it simultaneously — they scribble over each other and the number comes out wrong. That's a race condition. The fix is: don't keep a running number on the shared whiteboard. Keep beans stateless — no changing fields. If you truly need a shared counter, use a thread-safe tool like AtomicInteger.

Say this in the interview: That field is shared mutable state on a singleton, so concurrent access causes a **race condition** — two increments can overwrite each other. The fix is to keep the bean **stateless**; if shared state is truly needed, use thread-safe constructs like AtomicInteger or move the state out of the singleton.

Q3. Two beans implement the same interface, injected without a qualifier. What happens? How to fix?

In plain words: You asked for 'a payment method' but there are two on the shelf — Spring freezes because it doesn't know which to give you, and the app won't even start. Two fixes: put a 'default' sticker on one (@Primary), or name exactly the one you want (@Qualifier). If you use both, the specific name wins.

Say this in the interview: Spring throws **NoUniqueBeanDefinitionException** at startup because it can't decide which bean to inject. I fix it with **@Primary** to set a default, or **@Qualifier** to name the specific bean. @Qualifier beats @Primary when both are present.

Q4. Difference between @Component and @Bean? When must you use @Bean?

In plain words: @Component is like auto-hiring anyone wearing your company badge — Spring finds your own classes automatically. @Bean is like writing a manual offer letter for an outside contractor who has no badge. You **MUST** use @Bean when the class comes from a library you didn't write — you can't add a badge to someone else's code, so you create it by hand.

Say this in the interview: @Component is for classes I own — Spring auto-detects them via component scanning. @Bean is a method in a @Configuration class that creates the bean programmatically. I **must** use @Bean for a **third-party class** I can't annotate, like ObjectMapper or a RestTemplate, or when creation needs custom logic.

Q5. Is @Repository functionally different from @Component, or just a label?

In plain words: It's not just a label. Besides marking the class as a bean, @Repository has a real bonus: it automatically translates ugly, database-specific error messages into one clean, consistent set of Spring exceptions. So your code doesn't have to care whether the database was MySQL or Oracle — the errors look the same. @Service and @Controller, on the other hand, are mostly just labels for readability.

Say this in the interview: Not just a label. @Repository is a specialization of @Component, so it creates a bean, but it also enables **exception translation** — it converts vendor-specific persistence exceptions like SQLException into Spring's consistent DataAccessException hierarchy. @Service and @Controller are currently mostly semantic labels.

Section 2 • Container, Auto-Config & Startup

Q6. What is the ApplicationContext?

In plain words: It's the manager of the warehouse where all your objects (beans) live. It builds them, connects them to each other, hands them out when asked, and manages them from birth to shutdown. Whenever you need something, you ask the manager instead of digging through shelves yourself. It IS the IoC container in action.

Say this in the interview: The ApplicationContext is Spring's **IoC container** — the object that creates all the beans, injects their dependencies, and manages their lifecycle. It's what holds and hands out beans throughout the app.

Q7. Difference between BeanFactory and ApplicationContext?

In plain words: BeanFactory is a basic vending machine — it gives you an item when you ask, nothing fancy. ApplicationContext is a full-service store: same items, PLUS extra staff and features like auto-configuration, announcements (events), multi-language support, annotation handling, and it stocks the shelves in advance (eager loading) instead of only when asked. In real apps we always use ApplicationContext.

Say this in the interview: ApplicationContext is a **superset of BeanFactory**. BeanFactory just creates and injects beans lazily. ApplicationContext adds auto-configuration, event publishing, internationalization, annotation support, and eagerly creates singletons at startup. In practice we always use ApplicationContext.

Q8. Is ApplicationContext the implementation of the IoC container?

In plain words: Careful — ApplicationContext is an interface (a contract saying 'a container can do these things'), not the actual implementation. The real implementation is a concrete class behind it, like AnnotationConfigApplicationContext. It's exactly like List vs ArrayList: List is the interface, ArrayList is the implementation.

Say this in the interview: ApplicationContext is the **interface** for Spring's IoC container — a superset of BeanFactory. The actual **implementation** is a concrete class; in Spring Boot it's typically an AnnotationConfigApplicationContext variant, created automatically at startup.

Q9. What is auto-configuration, and what is the 'classpath'?

In plain words: The classpath is everything currently in your kitchen — your own code plus every library you installed. Auto-configuration means Spring Boot peeks in the kitchen and sets things up based on what it finds: see a web library? It auto-sets-up a web server. You wrote no setup code. And if you bring your own version of something, it steps aside and uses yours.

Say this in the interview: The classpath is the set of all classes and library JARs available at runtime. Auto-configuration inspects the classpath and configures matching beans automatically using **@Conditional** annotations — e.g. the web starter triggers embedded Tomcat and JSON setup. It **backs off** if I define my own bean.

Q10. What happens when a Spring Boot app starts?

In plain words: You run main, which calls run(). Spring opens an empty warehouse (the context), walks around noting every badge-wearing class (component scan), auto-sets-up framework pieces based on your libraries (auto-config), then actually builds all the objects and wires them together. Finally, if it's a web app, it starts the Tomcat server on port 8080 and you're live.

Say this in the interview: It begins with **SpringApplication.run()**, which creates the **application context**, does **component scanning** to find beans, runs **auto-configuration** based on the classpath, then **creates and injects** all beans. If it's a web app it starts **embedded Tomcat on port 8080** and is ready to serve requests.

Q11. What three annotations does @SpringBootApplication combine?

In plain words: It's a bundle of three: @Configuration (this class can define beans), @ComponentScan (go find my beans), and @EnableAutoConfiguration (auto-set-up based on my libraries). One annotation, three jobs.

Say this in the interview: @SpringBootApplication combines **@Configuration**, **@ComponentScan**, and **@EnableAutoConfiguration**.

Section 3 • Scopes, Lifecycle & Configuration

Q12. What is the default bean scope? Are beans thread-safe?

In plain words: Default is singleton — ONE shared copy for the whole app, like a shared office printer. Because it's shared, it is NOT thread-safe if it stores changing data in fields — multiple users would clobber each other's data. Keep it stateless and you're fine.

Say this in the interview: The default scope is **singleton** — one shared instance per application context. Singletons are **not thread-safe if they hold mutable state**, since they're shared across threads. The fix is to keep beans stateless.

Q13. Singleton vs prototype scope?

In plain words: Singleton = one shared copy everyone uses (shared printer). Prototype = a brand-new copy every single time you ask (a fresh notepad handed to each person). Use prototype when each user genuinely needs their own separate instance with its own state.

Say this in the interview: Singleton (default) is one shared instance for the whole context. Prototype creates a **new instance every time it's requested**. I use prototype when each consumer needs its own separate stateful instance.

Q14. Explain the bean lifecycle. What do `@PostConstruct` and `@PreDestroy` do?

In plain words: A bean is born (constructor), gets its tools (injection), then gets a 'now set yourself up' moment (`@PostConstruct`) — good for loading data or opening connections. It lives and does its job. Right before the app shuts down, it gets a 'clean up before you leave' moment (`@PreDestroy`) — good for closing connections.

Say this in the interview: The order is: constructor → dependency injection → `@PostConstruct` → bean in use → `@PreDestroy` → destroyed. `@PostConstruct` runs once after injection for setup like caching or opening connections; `@PreDestroy` runs once before shutdown for cleanup.

Q15. What are profiles and why use them?

In plain words: Same app, different environments — your laptop, testing, and live production each need different settings (different databases, etc.). Profiles let you keep separate settings files per environment and switch with one line, so you never edit code to move between them. Like 'Work mode' vs 'Home mode' on your phone.

Say this in the interview: Profiles provide environment-specific configuration — separate application-{env}.properties files activated via **spring.profiles.active**. I've used `@Profile` to load different beans per environment, like separate datasource config.

Q16. `@Value` vs `@ConfigurationProperties`?

In plain words: `@Value` grabs one setting at a time — fine for a value here and there. `@ConfigurationProperties` scoops up a whole GROUP of related settings into one tidy object automatically. For many related settings, the group approach is cleaner, type-safe, and the professional choice.

Say this in the interview: For a single value I use `@Value`, but for a group of related properties I prefer `@ConfigurationProperties` — it's type-safe, cleaner, supports validation, and binds a whole group at once instead of scattering `@Value` everywhere.

Section 4 • Circular Dependencies & AOP

Q17. What is a circular dependency and how do you fix it?

In plain words: Two objects each need the other to be built first — like two people each refusing to start until the other finishes. Spring gets stuck and the app won't start. Best fix: redesign so a third object holds the shared logic (usually the real problem). Quick workarounds: `@Lazy` (delay one until first used) or setter injection.

Say this in the interview: A circular dependency is when two beans depend on each other; with constructor injection it fails at startup with `BeanCurrentlyInCreationException`. The right fix is usually to **refactor** — extract the shared responsibility into a third class. `@Lazy` or setter injection are quick workarounds, but I treat it as a design smell.

Q18. What is Spring AOP and where is it used?

In plain words: AOP handles the 'everywhere' chores — logging, security, transactions — that would otherwise be copy-pasted into every method. You write the chore once and tell Spring to apply it automatically around the methods you choose. Like security cameras watching all rooms instead of a guard in each one.

Say this in the interview: AOP handles **cross-cutting concerns** like logging, security, and transactions — logic needed across many methods but separate from business logic. The classic example is `@Transactional`.

Q19. How does `@Transactional` work under the hood?

In plain words: When you tag a method `@Transactional`, Spring secretly wraps your object in a stand-in (a proxy). That stand-in starts a database transaction before your method runs, and commits it if all goes well or rolls it back if something throws. You write none of that plumbing — the proxy does it via AOP.

Say this in the interview: Spring creates an **AOP proxy** around the bean. The proxy starts a transaction before the method, then commits on success or rolls back on exception — so I get transaction management just by adding the annotation.

Q20. Difference between `@Controller` and `@RestController`?

In plain words: `@Controller` traditionally returns the name of a web PAGE to show.

`@RestController` returns raw DATA (usually JSON) straight back to the caller — it's `@Controller` with automatic 'send data, not a page' behavior built in. For REST APIs, you use `@RestController`.

Say this in the interview: `@RestController` is `@Controller` + `@ResponseBody`. It returns data like JSON directly, while `@Controller` typically returns a view/page name. For REST APIs I use `@RestController`.

How to Use This Sheet

1. Read the **plain-words** part until the concept clicks.
2. Memorize the **green interview answer** — that's your spoken response.
3. For each, add your own **project story**: 'In my project we used this when...'. That personal detail is what proves 2.5 YOE experience to an interviewer.